

p0019r1 : Atomic View

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0019r1
Date: 2016-02-04
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Hans Boehm
Contact: hboehm@google.com
Author: Olivier Giroux
Contact: ogiroux@nvidia.com
Author: James Reus
Contact: reus1@llnl.gov
Audience: SG1 Concurrency
URL: <https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0019.rst>

1 Introduction

This paper proposes an extension to the atomic operations library [atomics] for atomic operations applied to non-atomic objects. The proposal is in five parts: (1) the concept of an atomic view, (2) application of this concept applied to single objects, (3) application of this concept applied to members of a very large array, and (4) motivating use cases and illustrative examples.

```
namespace std {  
namespace experimental {  
    template< class T > atomic_view ;  
    template< class T > atomic_array_view ;  
}}
```

Note: Feedback from SG1 Library Evolution Working Group (LEWG) on P0009r0, Polymorphic Multidimensional Array View, noted that the term *view* has the connotation of read-only. In response the P0009r0 *array_view* name has been revised to *array_ref* in P0009r1. The proposed names *atomic_view* and *atomic_array_view* may have the same feedback from LEWG, potentially resulting in a similar naming revision.

2 Atomic View Concept

A class conforming to the atomic view concept provides atomic operations for the viewed non-atomic object. These operations replicate the operations on `std::atomic<T>` [29.5, Atomic types]; however, operations are const with respect to the atomic view object in contrast to the non-const and volatile operations of a `std::atomic<T>` object. An atomic view does not own (neither exclusively nor shared) the viewed non-atomic object.

The following *atomic-view-concept* specification is included to define requirements for classes conforming to the *atomic-view-concept* and does not imply the existence of a template class of this name.

```
template< class T >  
struct atomic-view-concept {  
    // Functionality matches corresponding member of atomic<T>  
    static constexpr bool is_always_lock_free = implementation defined ;  
    bool is_lock_free() const noexcept;  
    void store( T , memory_order = memory_order_seq_cst ) const noexcept;  
    T load( memory_order = memory_order_seq_cst ) const noexcept;  
    operator T() const noexcept ;  
    T exchange( T , memory_order = memory_order_seq_cst ) const noexcept;  
    bool compare_exchange_weak( T& , T , memory_order , memory_order ) const noexcept;  
    bool compare_exchange_strong( T& , T , memory_order , memory_order ) const noexcept;  
    bool compare_exchange_weak( T& , T , memory_order = memory_order_seq_cst ) const noexcept;  
    bool compare_exchange_strong( T& , T , memory_order = memory_order_seq_cst ) const noexcept;
```

T operator=(T) const noexcept ;

// Functionality that may exist and deviate from corresponding member of atomic<T>, if any.

atomic-view-concept ();

atomic-view-concept (const *atomic-view-concept* &);

atomic-view-concept (*atomic-view-concept* &&);

atomic-view-concept & operator = (const *atomic-view-concept* &);

atomic-view-concept & operator = (*atomic-view-concept* &&);

constexpr bool operator() const noexcept;

};

Requires: Type *T* is trivially copyable.

Constructors and assignment operators of an *atomic-view-concept* may acquire shared resources such as concurrent locks to support atomic operations on the non-atomic object. and

constexpr bool operator() const noexcept ;

Returns if the *atomic-view-concept* object wraps a non-null pointer. A default constructed *atomic-view-concept* object returns false.

A class conforming to the atomic view concept shall provide the following operations when *T* is an integral type. These operations replicate the operations on *std::atomic<integral>* [29.5, Atomic types]; however, operations are const with respect to the atomic view object in contrast to the non-const and volatile operations of a **std::atomic<integral>** object.

template<> struct *atomic-view-concept* < *integral* > {

integral fetch_add(*integral* , memory_order = memory_order_seq_cst) const noexcept;

integral fetch_sub(*integral* , memory_order = memory_order_seq_cst) const noexcept;

integral fetch_and(*integral* , memory_order = memory_order_seq_cst) const noexcept;

integral fetch_or(*integral* , memory_order = memory_order_seq_cst) const noexcept;

integral fetch_xor(*integral* , memory_order = memory_order_seq_cst) const noexcept;

integral operator++(int) const noexcept;

integral operator--(int) const noexcept;

integral operator++() const noexcept;

integral operator--() const noexcept;

integral operator+=(*integral*) const noexcept;

integral operator-=(*integral*) const noexcept;

integral operator&=(*integral*) const noexcept;

integral operator|=(*integral*) const noexcept;

integral operator^=(*integral*) const noexcept;

};

Note that for consistency the *atomic-view-concept<integral>* mathematical operator overloads retain the same mathematical inconsistency with respect to the mathematical operators for the *integral* type, as illustrated below.

```
int i(0);
++( ++i );           // ++i returns an lvalue
( i += 1 ) += 2 ;    // i+= returns an lvalue

std::atomic<int> ai(0);
++( ++( ai ) );      // error: ++ai returns an rvalue
( ai += 1 ) += 2 ;    // error: ai+= returns an rvalue
```

3 Atomic View for a Single Object

An **atomic_view<T>** object is used to perform atomic operations on the globally accessible viewed non-atomic object. The intent is for **atomic_view<T>** to provide the best-performing implementation of *atomic-view-concept* operations for the type *T*. All atomic operations on an instance of **atomic_view<T>** are atomic with respect to any other instance that views the same globally accessible object, as defined by equality of pointers to that object.

[Note: Introducing concurrency within legacy codes may require replacing operations on existing non-atomic objects with atomic operations such that the non-atomic object cannot be replaced with a *std::atomic* object. - end note]

[Note: An object may be heavily used non-atomically in well-defined phases of an application. Forcing such objects to be exclusively *std::atomic* would incur an unnecessary performance penalty. - end note]

```
template< class T > struct atomic_view { // conforms to atomic view concept

    explicit atomic_view( T & ); // wrapping constructor is NOT noexcept

    atomic_view();
    atomic_view( atomic_view && ) noexcept ;
    atomic_view( const atomic_view & ) noexcept ;
    atomic_view & operator = ( atomic_view && ) noexcept ;
    atomic_view & operator = ( const atomic_view & ) noexcept ;
    ~atomic_view() noexcept ;
};
```

Requires: Type *T* is trivially copyable.

[Note: The intent is for atomic operations of *atomic_view<T>* to directly update the referenced object. The *wrapping constructor* may acquire a resource, such as a lock from a collection of address-sharded locks, to perform atomic operations. When such a resource is necessary subsequent copy and move constructors and assignment operators may reduce overhead by copying or moving the previously acquired resource as opposed to re-acquiring that resource. – end note]

atomic_view<T>::atomic_view();

Effects: This instance does not reference a globally accessible object.

atomic_view<T>::atomic_view(T & obj);

Requires: The referenced obj must be properly aligned for its type T, otherwise behavior is undefined.

Requires: While any *atomic_view* instance exists for a referenced object all accesses of that object shall occur through an *atomic_view* referencing that object, otherwise behavior is undefined. Multiple instances of an *atomic_view* may be constructed with the wrapping constructor which reference the same globally accessible object.

Effects: This wrapping constructor wraps the globally accessible referenced object. Atomic operations on this instance are atomic with respect to atomic operations on any **atomic_view** instance that reference the same globally accessible object. [Note: This constructor is allowed to throw an exception if the referenced object is not properly aligned. - end note] [Note: This constructor may obtain a resource as necessary to support atomic operations. This constructor is allowed to throw an exception if such a resource could not be obtained. – end note]

Effects: All accesses of the wrapped object in the same execution context that appear before the wrapping constructor shall happen before the wrapping constructor completes. [Note: As if a memory fence were performed on the on the wrapped object within the wrapping constructor. - end note]

```
void foo( int & i ) {
    i = 42 ;
    atomic_view<int> ai(i);
    std::async( [=]() { assert( ai.load() == 42 ); });
}
```

atomic_view<T>::atomic_view(atomic_view && rhs) noexcept ;
atomic_view<T>::atomic_view(const atomic_view & rhs) noexcept ;
atomic_view<T> & atomic_view<T>::operator = (atomic_view && rhs) noexcept ;
atomic_view<T> & atomic_view<T>::operator = (const atomic_view & rhs) noexcept ;

Effects: If *rhs* references a globally accessible object then this instance references the same object otherwise this instance does not reference a globally accessible object. [Note: If *rhs* holds a resource to support atomic operations then that resource should be copied or moved as appropriate. - end note]

atomic_view<T>::~~atomic_view() noexcept ;

Effects: This instance does not reference a globally accessible object. [Note: If this instances holds a resource to support atomic operations then that resource should be released or destroyed as appropriate. - end note]

4 Atomic View for a Very Large Array

High performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting HPC pattern.

An **atomic_array_view** object is used to perform atomic operations on the viewed non-atomic members of the array. The intent is for **atomic_array_view** to provide the best-performing implementation of atomic-view-concept operations for the members of the array.

```
template< class T > struct atomic_array_view {

    static constexpr bool is_always_lock_free = implementation defined ;
    bool is_lock_free() const noexcept ;

    // Returns true if the view wraps an array and member access is valid.
    explicit bool operator() const noexcept ;

    atomic_array_view( T * , size_t ); // Wrapping constructor is NOT noexcept
    atomic_array_view() noexcept ;
    atomic_array_view( atomic_array_view && ) noexcept ;
    atomic_array_view( const atomic_array_view & ) noexcept ;
    atomic_array_view & operator = ( atomic_array_view && ) noexcept ;
    atomic_array_view & operator = ( const atomic_array_view & ) noexcept ;
    ~atomic_array_view() noexcept ;

    size_t size() const noexcept ;

    using reference = implementation-defined-atomic-view-concept-type ;

    reference operator[] ( size_t ) const noexcept ;
};
```

Requires: Type *T* is trivially copyable.

[Note: The *wrapping constructor* may acquire resources, such as a set of locks, to perform atomic operations. When such a resource is necessary subsequent copy and move constructors and assignment operators may reduce overhead by copying or moving the previously acquired resource as opposed to re-acquiring that resource. The intent is to enable amortization of the time and space overhead of obtaining and releasing such resources. – end note] – end note]

using reference = *implementation-defined-atomic-view-concept-type* ;

The **reference** type conforms to *atomic-view-concept* for type *T*.

```
static constexpr bool is_always_lock_free = implementation defined ;
bool atomic_array_view<T>::is_lock_free() const noexcept ;
```

Effects: Whether atomic operations on members are (always) lock free.

```
atomic_array_view<T>::atomic_array_view( T * ptr , size_t N );
```

Requires: The array referenced by [ptr .. ptr+N-1] must be properly aligned for its type *T*, otherwise behavior is undefined.

Requires: While any *atomic_array_view* instance exists for a referenced array all accesses of that array and its members shall occur through an *atomic_array_view* referencing that array, otherwise behavior is undefined. Multiple concurrent instances of an *atomic_array_view* may be constructed with the wrapping constructor which references any of the any globally accessible members (i.e., has an overlapping range) if-and-only-if the *atomic_array_view* type **is_always_lock_free**. If NOT **is_always_lock_free** then construction of multiple concurrent instances via the wrapping constructor has undefined behavior. [Note: This allows a non-lock-free *atomic_array_view* to acquire a set of locks that are exclusively associated with the wrapped array. - end note]

Effects: The *wrapping constructor* wraps the referenced globally accessible array [ptr .. ptr+N-1]. Atomic operations on members of this instance are atomic with respect to atomic operations on members any **atomic_array_view** instance

that references the same globally accessible array. [Note: This constructor is allowed to throw an exception if the referenced array is not properly aligned. - end note] [Note: This constructor may obtain resources as necessary to support atomic operations on members. This constructor is allowed to throw an exception if such resources could not be obtained. – end note]

Effects: All accesses of the wrapped array members in the same execution context that appear before the wrapping constructor shall happen before the wrapping constructor completes. [Note: As if a memory fence were performed on the wrapped array within the wrapping constructor. - end note]

```
void foo( int * i , size_t N ) {
    i[0] = 42 ;
    i[N-1] = 42 ;
    atomic_array_view<int> ai(i,N);
    std::async( [=]()
    {
        assert( ai[0].load() == 42 );
        assert( ai[N-1].load() == 42 );
    });
}
```

```
atomic_array_view<T>::atomic_array_view( atomic_array_view && rhs ) noexcept ;
atomic_array_view<T>::atomic_array_view( const atomic_array_view & rhs ) noexcept ;
atomic_array_view<T> & atomic_array_view<T>::operator = ( atomic_array_view && rhs ) noexcept ;
atomic_array_view<T> & atomic_array_view<T>::operator = ( const atomic_array_view & rhs ) noexcept ;
```

Effects: If *rhs* references a globally accessible array then this instance references the same array otherwise this instance does not reference a globally accessible object. [Note: If *rhs* holds resources to support atomic operations then that resource should be copied or moved as appropriate. It may be appropriate for these resources to be managed with *std::shared_ptr* semantics. - end note]

```
atomic_array_view<T>::~~atomic_array_view() noexcept ;
```

Effects: This instance does not reference a globally accessible object. [Note: If this instances holds resources to support atomic operations then those resources should be released or destroyed as appropriate. - end note]

```
atomic_array_view<T>::reference atomic_array_view<T>::operator[]( size_t i ) const noexcept ;
```

Requires: $i < N$. The program is ill-formed if *i* is out of bounds.

Requires: The returned **reference** object must be destroyed or re-assigned before the last associated *atomic_array_view* instance is destroyed, otherwise behavior is undefined.

Returns: An instance of **reference** type for the *i*-th member of the **atomic_array_view**, where indexing is zero-based. [Note: The intent is for efficient generation of the returned object with respect to obtaining a resource, such as a shared locking mechanism, that may be required to support atomic operations on the referenced member. – end note]

5 Notes and Examples

5.1 Atomic Array View

Under the HPC use case the member access operator, proxy type constructor, or proxy type destructor will be frequently invoked; therefore, an implementation should trade off decreased overhead in these operations versus increased overhead in the wrapper constructor and final destructor.

Usage Scenario for **atomic_array_view<T>**

- A very large array of trivially copyable members is allocated.
- A parallel algorithm initializes members through non-conflicting assignments.
- The array is wrapped by an **atomic_array_view<T>**.
- One or more parallel algorithms update members of the array through atomic view operations.
- The **atomic_array_view<T>** is destructed.
- Parallel algorithms access array members through non-conflicting reads, writes, or updates.

Example:

```
// atomic array view wrapper constructor:
atomic_array_view<T> array( ptr , N );
```

```
// atomic operation on a member:
array[i].atomic-operation(...);

// atomic operations through a temporary value
// within a concurrent function:
atomic_array_view<T>::reference x = array[i];
x.atomic-operation-a(...);
x.atomic-operation-b(...);
```

Possible interface for **atomic_array_view<T>::reference**

```
struct implementation-defined-proxy-type {    // conforms to atomic view concept

    // Construction limited to move
    implementation-defined-proxy-type(implementation-defined-proxy-type && ) = noexcept ;
    ~implementation-defined-proxy-type();

    implementation-defined-proxy-type() = delete ;
    implementation-defined-proxy-type( const implementation-defined-proxy-type & ) = delete ;
    implementation-defined-proxy-type &
        operator = ( const implementation-defined-proxy-type & ) = delete ;
};
```

Wrapping constructor options for **atomic_array_view<T>**

A wrapping constructor of the form (T*begin, T*end) could be valid. However, the (T*ptr, size_t N) version is preferred to minimize potential confusion with construction from non-contiguous iterators. Wrapping constructors for standard contiguous containers would also be valid. However, such constructors could have potential confusion as to whether the atomic_array_view would or would not track resizing operations applied to the input container.

Implementation note for **atomic_array_view<T>**

All non-atomic accesses of array members that appear before the wrapping constructor must happen before subsequent atomic operations on the atomic_array_view members. For example:

```
void foo( int * i , size_t N ) {
    i[0] = 42 ;
    i[N-1] = 42 ;
    atomic_array_view<int> ai(i,N);
    // Operations on 'i' must happen before operations on 'ai'
    foreach( parallel_policy, 0, M, [=]( int j ){ ++ai[j*N] ; } );
}
```

5.2 Mathematically Consistent Integral Operator Overloads

As previously noted the **std::atomic<integral>** mathematical operator overloads are inconsistent with the mathematical operators for *integral*. The *atomic-view-concept<integral>* retains these inconsistent operator overloads. Consistent mathematical operator semantics would be restored with the following operator specifications. However, such a change would break backward compatibility and is therefore only noted and not a proposed change.

```
template<> struct atomic < integral > {
```

```
    volatile atomic & operator++(int) volatile noexcept ;
    atomic & operator++(int) noexcept ;
    volatile atomic & operator--(int) volatile noexcept ;
    atomic & operator--(int) noexcept ;
```

```
// fetch-and-increment, fetch-and-decrement operators:
integral operator++() volatile noexcept ;
integral operator++() noexcept ;
integral operator--() volatile noexcept ;
integral operator--() noexcept ;
```

```
volatile atomic & operator+=( integral ) volatile noexcept;
atomic & operator+=( integral ) noexcept;
volatile atomic & operator-=( integral ) volatile noexcept;
atomic & operator-=( integral ) noexcept;
volatile atomic & operator&=( integral ) volatile noexcept;
atomic & operator&=( integral ) noexcept;
```

```
volatile atomic & operator|=( integral ) volatile noexcept;  
atomic & operator|=( integral ) noexcept;  
volatile atomic & operator^=( integral ) volatile noexcept;  
atomic & operator^=( integral ) noexcept;  
};
```

```
template<> struct atomic-view-concept < integral > {
```

```
    const atomic-view-concept & operator++(int) const noexcept;  
    const atomic-view-concept & operator--(int) const noexcept;
```

```
    integral operator++() const noexcept;  
    integral operator--() const noexcept;
```

```
    const atomic-view-concept & operator+=( integral ) const noexcept;  
    const atomic-view-concept & operator-=( integral ) const noexcept;  
    const atomic-view-concept & operator&=( integral ) const noexcept;  
    const atomic-view-concept & operator|=( integral ) const noexcept;  
    const atomic-view-concept & operator^=( integral ) const noexcept;
```

```
};
```